

Qualidade de Código

Testes Unitários, Automação de Processos e Gerenciamento de Dependências



Testes Unitários



Automação



Dependências

Roteiro da aula



O que é qualidade de código

01

Por que investir tempo em qualidade e o custo de não fazê-lo.



Testes unitários

02

Conceito, pirâmide de testes, TDD e boas práticas.



Escrevendo testes na prática

03

Exemplos com JavaScript e Jest; cobertura de código.



Automação de processos

04

Scripts npm, linters, formatadores, hooks e CI/CD.



Gerenciamento de dependências

05

npm, semver, lockfiles, auditoria e segurança.



Fechamento e atividade

06

Síntese, checklist de qualidade e exercício prático.

O que é "qualidade de código"?

Código de qualidade não é apenas código que funciona — é código que continua funcionando, é fácil de entender, de modificar e de confiar ao longo do tempo.



Confiabilidade

Faz o que deveria e resiste a mudanças sem quebrar.



Manutenibilidade

Legível e organizado — qualquer pessoa da equipe entende.



Reprodutibilidade

Build, testes e dependências iguais em qualquer máquina.



Evolutibilidade

Aceita novas features sem virar uma bola de neve de bugs.

O custo de ignorar a qualidade

Quanto mais tarde um bug é descoberto, mais caro ele fica. Corrigir na produção pode custar dezenas de vezes mais do que evitar com um teste.

Regra 1-10-100

1×

Corrigir na fase de desenvolvimento

10×

Corrigir durante os testes/QA

100×

Corrigir depois, já em produção



Dívida técnica

Atalhos de hoje viram retrabalho caro amanhã.



Lentidão da equipe

Medo de mexer no código trava novas entregas.



Bugs em produção

Falhas atingem usuários reais e a reputação.

Testes unitários

Um teste unitário verifica, de forma isolada e automática, a menor parte testável do sistema — normalmente uma função ou método.



Rápidos

Executam em milissegundos; rodam a cada alteração.



Isolados

Não dependem de banco, rede ou de outros testes.



Repetíveis

Mesmo resultado sempre — sem passar "às vezes".



Automáticos

Afirmam sozinhos se passaram ou falharam.

Padrão AAA

A estrutura clássica de um bom teste:

1

Arrange

Prepare os dados e o cenário do teste.

2

Act

Execute a ação/função que será testada.

3

Assert

Verifique se o resultado é o esperado.

A pirâmide de testes

Uma estratégia saudável tem muitos testes rápidos e baratos na base, e poucos testes lentos e caros no topo.



Por que essa forma?

- Testes unitários dão feedback em segundos e apontam exatamente onde está o erro.
- Testes de integração garantem que as peças conversam entre si.
- Testes end-to-end validam o fluxo completo do usuário, mas são frágeis e lentos.
- Invertendo a pirâmide (o "cone de sorvete"), a suíte fica lenta e instável.

TDD — Test-Driven Development

Uma disciplina em que o teste vem antes do código. Um ciclo curto que se repete a cada pequena funcionalidade.



1. RED

Escreva um teste que falha

Descreva o comportamento esperado antes de existir o código.



2. GREEN

Faça o teste passar

Implemente o mínimo necessário para o teste ficar verde.



3. REFACTOR

Melhore o código

Limpe e organize com a segurança de que os testes protegem.

Na prática: um teste com Jest

Jest é um framework popular de testes para JavaScript/TypeScript. Veja o código e o teste correspondente.

carrinho.js

```
function calcularTotal(itens) {
  return itens.reduce(
    (soma, i) => soma + i.preco * i.qtd,
    0
  );
}

module.exports = { calcularTotal };
```

carrinho.test.js

```
const { calcularTotal } =
  require('./carrinho');

test('soma preco x quantidade', () => {
  // Arrange
  const itens = [
    { preco: 10, qtd: 2 },
    { preco: 5, qtd: 3 },
  ];
  // Act
  const total = calcularTotal(itens);
  // Assert
  expect(total).toBe(35);
});
```

Cobertura de código (coverage)

Mede quanto do código é exercitado pelos testes. É um indicador útil — mas não uma garantia de qualidade.



Linhas

Quais linhas foram executadas.



Branches

Se ambos os caminhos de um if/else rodaram.



Funções

Quais funções foram chamadas.



Cuidado com a métrica

100% de cobertura não significa zero bugs. Cobrir código não é o mesmo que testar bons cenários e casos-limite.

70–80%

faixa de cobertura frequentemente adotada como meta pragmática — priorize o código crítico.

PARTE 2

Automação de Processos

"Se você faz algo mais de duas vezes, automatize."



Automatizando o dia a dia



Scripts npm

Comandos padronizados em package.json: npm test, npm run lint, npm run build.



Linters (ESLint)

Analizam o código e apontam erros e más práticas automaticamente.



Formatadores (Prettier)

Padronizam o estilo do código, encerrando debates sobre formatação.



Git Hooks (Husky)

Rodam testes e linters antes de cada commit ou push.

Integração e entrega contínuas (CI/CD)

A cada push, um servidor (ex.: GitHub Actions) roda automaticamente a esteira que valida e publica o código.



Benefício: problemas são detectados cedo, o feedback é rápido e a publicação vira uma rotina previsível, não um evento de risco.



PARTE 3

Gerenciamento de Dependências

Todo projeto moderno é construído sobre código de terceiros.
Saber gerenciá-lo é essencial para estabilidade e segurança.

package.json e versionamento semântico

package.json

```
{
  "name": "loja-web",
  "version": "1.4.2",
  "scripts": {
    "test": "jest",
    "lint": "eslint ."
  },
  "dependencies": {
    "express": "^4.18.0"
  },
  "devDependencies": {
    "jest": "~29.7.0"
  }
}
```

SemVer: MAJOR.MINOR.PATCH

1

MAJOR

Mudanças incompatíveis — pode quebrar seu código.

4

MINOR

Novas funcionalidades compatíveis com o anterior.

2

PATCH

Correções de bugs compatíveis.

^ aceita atualizações MINOR e PATCH • ~ aceita apenas PATCH

Lockfiles, auditoria e segurança



package-lock.json

Registra a versão exata de cada dependência (e das sub-dependências), garantindo builds idênticos em toda máquina. Sempre versiona o lockfile.



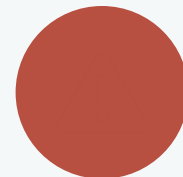
npm audit

Verifica as dependências contra um banco de vulnerabilidades conhecidas e sugere correções com npm audit fix.



Deps transitivas

Suas dependências têm dependências. Uma falha lá embaixo afeta seu projeto — por isso auditar toda a árvore importa.



Boas práticas

Atualize com regularidade, evite pacotes abandonados, fixe versões em produção e revise o que você instala.

Checklist de qualidade do projeto



A suíte de testes unitários cobre a lógica de negócio crítica.



Os testes rodam rápido e passam de forma consistente.



Há linter e formatador configurados e padronizados.



Uma esteira de CI valida cada push automaticamente.



O lockfile está versionado no repositório.



As dependências são auditadas contra vulnerabilidades.

Atividade prática

Em duplas, aplique os três pilares da aula em um pequeno projeto Node.js.



1. Testar

Escreva uma função utilitária (ex.: validar CPF ou calcular desconto) e cubra-a com pelo menos 3 testes no padrão AAA usando Jest.



2. Automatizar

Configure scripts npm para test e lint, e adicione o ESLint. Bônus: um hook que roda os testes antes do commit.



3. Depender bem

Adicione uma dependência, explique sua faixa de versão no package.json e rode npm audit, comentando o resultado.

Síntese

Qualidade é uma prática, não um acidente.



Testes dão confiança para mudar o código sem medo.



Automação elimina o trabalho manual e detecta erros cedo.



Dependências bem geridas trazem estabilidade e segurança.

Obrigado! Perguntas?